Robust Regression on Baseball Cards

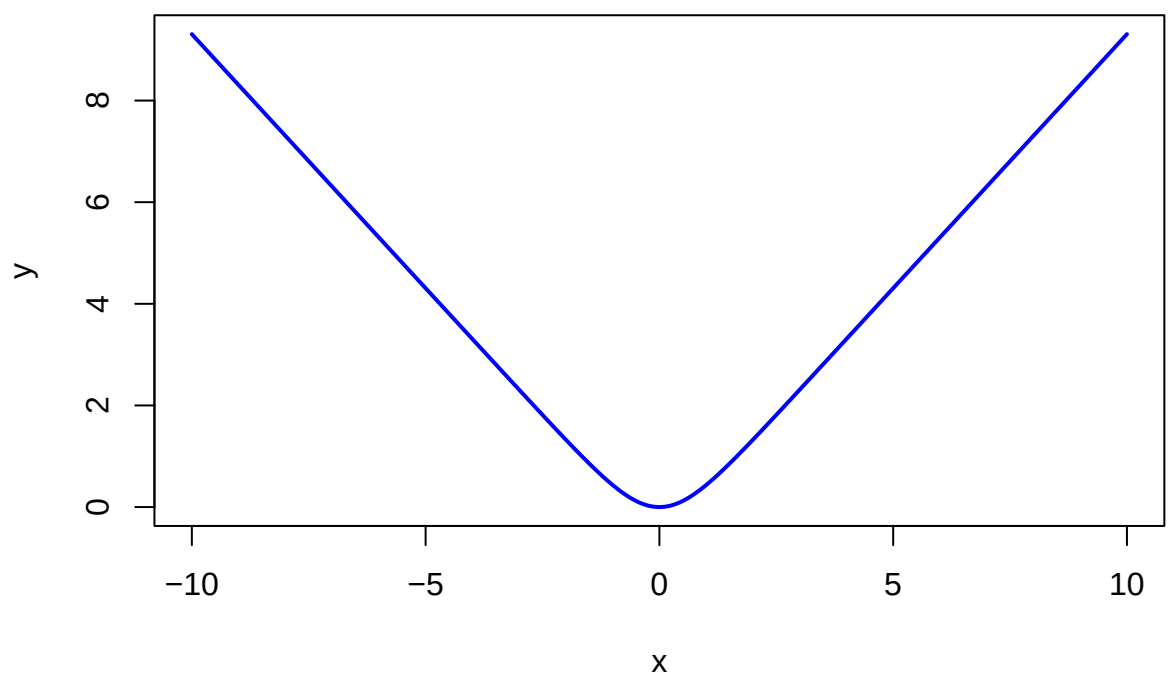
In "A Comprehensive Survey of Loss Functions in Machine Learning", by Qi Wang, Yue Ma, Kun Zhao & Yingjie Tian, available at https://link.springer.com/article/10.1007/s40745-020-00253-5, one alternative to Huber loss that is discussed is Log-cosh loss, defined as

$$\rho(\mathcal{P}, u) = \sum_{u \in \mathcal{P}} (\log(\cosh(r_u)))$$

where r_u is the residual $y_u - \hat{y}_u$, and cosh is the hyperbolic cosine.

a) (2 marks) Plot `log(cosh(x))` and comment on its appropriateness as a loss function.

```
x <- seq(-10, 10, length.out = 1000)
y <- log(cosh(x))
plot(x, y, type = "l", col = "blue", lwd = 2)
```



It is appropriate for a loss function since it is a convex function and clearly has a global minimum.

b) (1 mark) Write a function of log-cosh loss called `logcoshfn`.

$$\rho(\hat{\theta}) = \sum_{u \in \mathcal{P}} \log(\cosh(y_u - \alpha - \beta(x - \bar{x}))),\ \theta = (\alpha, \beta)$$

```
cards = read.csv("data/df_cards.csv")

createlogcoshRho <- function(x, y) {
  ## local variable
  xbar <- mean(x)
  ## Return this function
  function(theta) {
    alpha <- theta[1]
    beta <- theta[2]
    sum( log(cosh(y - alpha - beta * (x - xbar))) )
  }
}

logcoshfn = createlogcoshRho(cards$war, cards$n)
```

c) (3 marks) Find the derivative, write it **in LaTeX**

$$\frac{d}{dx}\left[\log(\cosh(x))\right] = \frac{\sinh(x)}{\cosh(x)} = \tanh(x)$$

$$\rho(\hat{\theta}) = \sum_{u \in \mathcal{P}} \log(\cosh(y_u - \alpha - \beta(x - \bar{x}))), \quad \theta = (\alpha, \beta)$$

$$g = \nabla\rho(\theta) = \begin{bmatrix} \frac{d\rho}{d\alpha} \\ \frac{d\rho}{d\beta} \end{bmatrix} = \begin{bmatrix} \sum_{u \in \rho} -\tanh\left(y_u - \alpha - \beta(x - \bar{x})\right) \\ \sum_{u \in \rho} -(x - \bar{x})\tanh\left(y_u - \alpha - \beta(x - \bar{x})\right) \end{bmatrix} = \begin{bmatrix} \sum_{u \in \rho} -\tanh\left(r_u\right) \\ \sum_{u \in \rho} -(x - \bar{x})\tanh\left(r_u\right) \end{bmatrix}$$

d) (1 mark) Write a function of the derivative of log-cosh loss called `logcoshprimefn`.

```
createlogcoshGradient <- function(x, y) {
  ## local variables
  xbar <- mean(x)
  ybar <- mean(y)
  function(theta) {
    alpha <- theta[1]
    beta <- theta[2]
    ru = y - alpha - beta*(x - xbar)
    -1*c( sum(tanh(ru)),
          sum(tanh(ru)*(x-xbar)) )
  }
}

logcoshprimefn = createlogcoshGradient(cards$war, cards$n)
```

e) (5 marks) Use the `logcoshfn` and `logcoshprimefn` functions as `rho` and `gradient` in the `gradientdescent` function, and find a robust estimate of the number of cards published (`n`, y) as a function of the career wins above replacement (WAR) (`war` x) of a baseball player in `df_cards.csv`,

and use it to fit a line to the data and plot it. *(Note: WAR is a measure of how much a player has contributed to their team's success, compared to a 'replacement' player who would be someone brought up from a league below. It's possible to have a WAR below zero, but in this dataset, those cases have been removed. In short, higher WAR means a better player.)*

Use $\hat{\theta}_0 = (7, 2)$

```
gradientDescent <- function(theta = 0,
      rhoFn, gradientFn, lineSearchFn, testConvergenceFn,
      maxIterations = 100,
      tolerance = 1E-6, relative = FALSE,
      lambdaStepsize = 0.01, lambdaMax = 0.5 ) {

  converged <- FALSE
  i <- 0

  while (!converged & i <= maxIterations) {
    g <- gradientFn(theta) ## gradient
    glength <-  sqrt(sum(g^2)) ## gradient direction
    if (glength > 0) d <- g /glength

    lambda <- lineSearchFn(theta, rhoFn, d,
                lambdaStepsize = lambdaStepsize, lambdaMax = lambdaMax)

    thetaNew <- theta - lambda * d
    converged <- testConvergenceFn(thetaNew, theta,
                                   tolerance = tolerance,
                                   relative = relative)
    theta <- thetaNew
    i <- i + 1
  }

  ## Return last value and whether converged or not
  list(theta = theta, converged = converged, iteration = i, fnValue = rhoFn(theta))
}
```

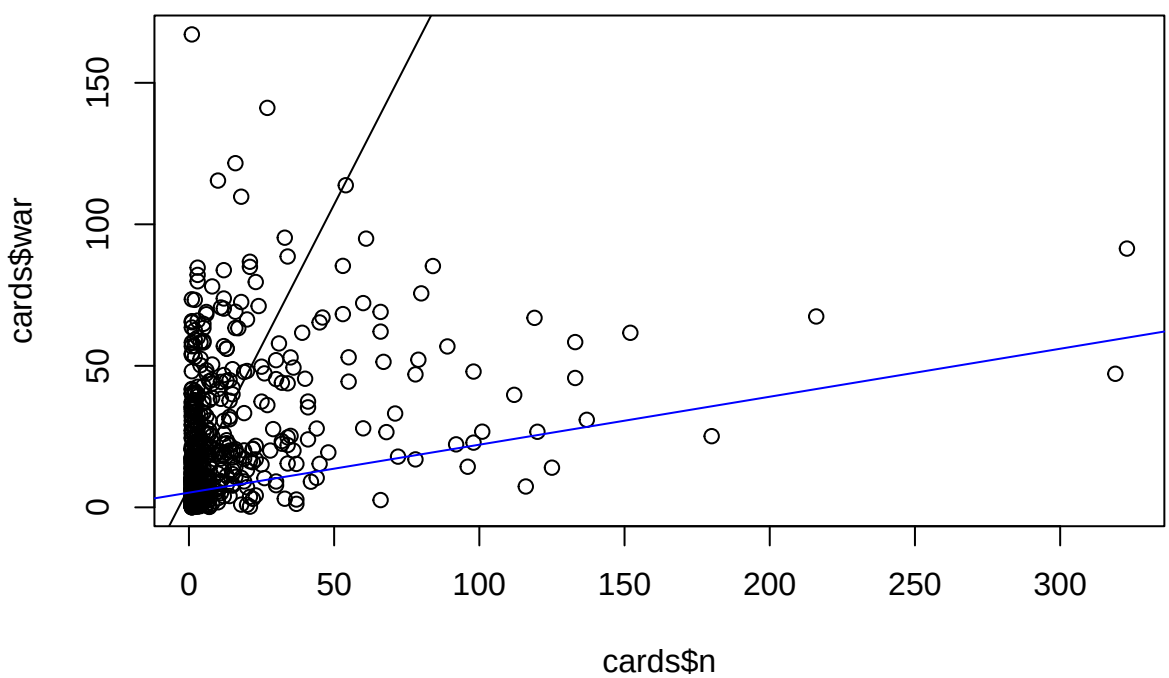
```
### line searching could be done as a simple grid search
gridLineSearch <- function(theta, rhoFn, d,lambdaStepsize = 0.01, lambdaMax = 1) {
  ## grid of lambda values to search
  lambdas <- seq(from = 0, by = lambdaStepsize,  to = lambdaMax)
  ## line search
  rhoVals <- sapply(lambdas, function(lambda) {rhoFn(theta - lambda * d)})
  ## Return the lambda that gave the minimum
  lambdas[which.min(rhoVals)]
}
```

```
### Where testCovergence might be (relative or absolute)
testConvergence <- function(thetaNew, thetaOld, tolerance = 1E-10, relative=FALSE) {
   sum(abs(thetaNew - thetaOld)) < if (relative) tolerance * sum(abs(thetaOld)) else tolerance
}
```

```r
gradientDescent(theta = c(7, 2), rhoFn = logcoshfn, gradientFn = logcoshprimefn,
                lineSearchFn = gridLineSearch, testConvergenceFn = testConvergence,
                lambdaStepsize = 0.001)
```

```
## $theta
## [1] 5.1895266 0.1694879
##
## $converged
## [1] TRUE
##
## $iteration
## [1] 70
##
## $fnValue
## [1] 6431.333
```

```r
plot(cards$n, cards$war)
abline(7, 2)
abline(5.1895266, 0.1694879, col = "blue")
```



f) (1 mark) Check your work using the `optim` function with the same initial values for `par` and `rho` as the `fn`.

```r
optim(par=c(7, 2), fn = logcoshfn)
```

```
## $par
## [1] 5.176915 0.168554
##
## $value
## [1] 6431.324
##
## $counts
## function gradient
##       71       NA
##
## $convergence
## [1] 0
##
## $message
## NULL
```